

11 — Embedding Generation: Batching & Retry Resilience

If you haven't read doc 06 yet, that one covers what an embedding actually is and the provider abstraction pattern (Gemini/Jina/local BGE) — this doc assumes that background and focuses on a different problem: how do we call an embedding provider *reliably* when we need thousands of embeddings, not just one?

What it is

`bilingual_etl/load/embeddings.py` wraps the raw embedding provider (`embed(texts)`) with two pieces of resilience logic:

1. **Batching** — instead of handing the provider one gigantic list of texts, we split it into small groups (50 texts at a time by default) and call the provider once per group.
2. **Retry-with-backoff** — if a single group's API call fails, we don't give up immediately. We wait, try again, and if it fails again we wait *longer* before the next try — up to a fixed number of attempts before finally giving up.

Neither of these change *what* an embedding is — they change *how safely* we get one when calling a real, sometimes-flaky, rate-limited API for thousands of pieces of text.

Real-life analogy

Imagine you need to photocopy 3,000 pages at a shop that only lets you feed 50 pages into the machine at a time, and the machine occasionally jams.

- **Batching** is feeding 50 pages at a time instead of trying to cram all 3,000 in at once (which the machine would reject outright, or choke on).
- **Retry-with-backoff** is: if the machine jams on a batch, you don't yank the handle immediately and again a second later — you wait a bit, try again, and if it jams again you wait even longer before the next attempt, because if the machine is overheating, hitting it faster and harder is more likely to break it than to fix it. After a few tries you give up on *that batch* and flag it, but you keep the 2,950 pages you already successfully copied — you don't throw away all your finished work just because page 2,340 had a problem.

Why we used it here

This project embeds text for 471 companies × 2 languages × several chunks each, plus roughly 3,000 categories × 2 languages — that's many thousands of individual texts needing embeddings (R9, vector path). The raw providers built in an earlier phase (`GeminiEmbedding`, `JinaEmbedding`, `LocalBGEEmbedding`) each expose a simple `embed(texts)` method, but two real risks show up the moment you call that method with thousands of texts at once:

- **Wasted work on failure.** If you send all 6,000+ texts in one call and it fails two-thirds of the way through (network blip, timeout, server error), you have nothing to show for it — you'd have to start completely over.

- **Rate limits.** Gemini and Jina are used here on their free tiers (per the multi-provider requirement R23), and free tiers cap how much you can send per request and per minute. An oversized single request is exactly the kind of thing that trips a rate limit or gets rejected outright.

Batching in groups of 50 keeps each request a reasonable size and means a failure only costs you that one batch's ~50 texts, not the other thousands already embedded successfully. Retry-with-backoff then handles the common case where a batch fails for a *transient* reason (a brief network hiccup, a momentary rate-limit rejection) by giving it a few more chances before treating it as a real failure.

Code walkthrough

From `bilingual_etl/load/embeddings.py`:

```
DEFAULT_BATCH_SIZE = 50
MAX_RETRIES = 3
INITIAL_BACKOFF_SECONDS = 2
```

Three tunable constants, not magic numbers scattered through the code: 50 texts per batch, up to 3 attempts per batch, starting with a 2-second wait.

Batching loop, in `embed_texts`:

```
for i in range(0, len(texts), batch_size):
    batch = texts[i : i + batch_size]
    results.extend(_embed_batch_with_retry(batch, provider))
```

`range(0, len(texts), batch_size)` steps through the list in jumps of 50 (0, 50, 100, ...). Each iteration slices out the next chunk of up to 50 texts, sends *that chunk* through the retry-wrapped call, and appends the returned vectors to the running `results` list. The overall function still looks to its caller like "give me a list of texts, get a list of vectors back" — batching is invisible from the outside.

This is verified by a real passing test (`tests/etl/test_embeddings.py::test_uneven_batches`): calling `embed_texts(["a", "b", "c"], provider=provider, batch_size=2)` results in exactly 2 calls to the mock provider — one with ["a", "b"], one with ["c"]. Three texts, batch size 2, produces a full batch followed by a leftover partial batch — exactly what the loop above should do.

Retry-with-backoff, in `_embed_batch_with_retry`:

```
def _embed_batch_with_retry(batch: list[str], provider: EmbeddingProvider) -> list[list[float]]:
    backoff = INITIAL_BACKOFF_SECONDS
    last_error: Exception | None = None

    for attempt in range(1, MAX_RETRIES + 1):
        try:
            return provider.embed(batch)
        except Exception as e:
            last_error = e
            logger.warning(f"Embedding batch failed (attempt {attempt}/{MAX_RETRIES}): {e}")
            if attempt < MAX_RETRIES:
                time.sleep(backoff)
                backoff *= 2

    raise RuntimeError(f"Embedding batch failed after {MAX_RETRIES} attempts: {last_error}") from last_error
```

Walking through it: attempt 1 calls `provider.embed(batch)`. If it succeeds, the function returns immediately — no waiting, no fuss. If it raises an exception, we log a warning, and (as long as we haven't used up all `MAX_RETRIES` attempts) sleep for `backoff` seconds, then **double backoff** for next time. So the wait sequence is 2 seconds, then 4 seconds. After the third attempt fails too, the loop ends and we raise a `RuntimeError` that wraps the last real error — the caller sees a clear, final failure instead of a silent gap in the data.

Why *double* the wait instead of retrying instantly or waiting the same fixed amount each time? If the API is genuinely rate-limited or briefly overloaded, retrying instantly (or at a constant short interval) means you keep hitting it at exactly the moment it's struggling — that can make the underlying problem worse (more rejected requests, longer recovery time) instead of better. Spacing retries out with a growing delay gives the API room to recover before you ask again. This pattern has a name you'll hear constantly in backend engineering: **exponential backoff**. It's not something invented for this project — it's a standard, widely-used resilience pattern anywhere software talks to an external API it doesn't fully control.

This is verified by a real passing test

(`tests/etl/test_embeddings.py::test_backoff_doubles_each_attempt`): a mock provider is set up to fail twice (`Exception("e1")`, `Exception("e2")`) and succeed on the third call. The test patches `time.sleep` and asserts the recorded sleep durations were exactly `[2, 4]` — proving the backoff really doubles, and that the third attempt actually recovers and returns a result.

Why mock `time.sleep` in the test at all? Without mocking it, this test would need to *actually wait* $2 + 4 = 6$ real seconds every time the test suite runs, just to prove a doubling calculation is correct. Mocking replaces the real sleep with a fake one that returns instantly but still records what it *would have* waited — so the test runs in milliseconds while still verifying the real timing logic.

Why this isn't duplicated in every provider. `GeminiEmbedding`, `JinaEmbedding`, and `LocalBGEEmbedding` each only know how to format a request and parse a response for *their own* API — that's already a full job (see doc 06). Batching and retry-with-backoff are a completely different concern: "how do I resiliently call any provider," regardless of which one it is. Putting that logic in `embeddings.py` as a thin wrapper around the provider interface means it's written once, tested once, and applies uniformly no matter which provider the Admin UI has selected — instead of three near-identical copies of the same retry loop silently drifting out of sync as the code evolves.

Why no new dependency. A popular, more feature-rich retry library like `tenacity` exists and is common in production Python code. It wasn't added here — the retry logic needed for this project (fixed max attempts, simple doubling delay) is about 10 lines of plain Python using nothing but the built-in `time.sleep`. Pulling in a new dependency for something this small would add a maintenance/compatibility surface for no real benefit; a hand-written loop this simple is easier to read, test, and reason about than learning another library's configuration surface.

How to explain this in an interview/to a teammate

"When we needed to embed thousands of texts through a free-tier API, we didn't just fire them all off in one call — we batched them into groups of 50 so a failure only costs you that batch's worth of work, not everything you'd already embedded. On top of that, each batch call is wrapped in a retry loop that uses exponential backoff: if a call fails, we wait 2 seconds and try again, then 4 seconds, then give up after 3 total attempts — doubling the wait instead of retrying instantly avoids hammering an API that's already

rate-limited or struggling. We kept that logic in one small wrapper module rather than copying it into each provider class, so 'how do I resiliently call any provider' stays separate from 'how do I talk to this specific API.' We tested it with mocked providers and a mocked `time.sleep`, so the tests verify the real retry/backoff behavior without the test suite actually taking six seconds to run."